Code:

```
# ==========================================
# 1. IMPORT LIBRARIES & SETUP
# ==========================================
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor
from xgboost import XGBRegressor
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

import warnings
warnings.filterwarnings('ignore')

# Set aesthetic visual style
sns.set_theme(style="whitegrid")
plt.rcParams['font.size'] = 11

# ==========================================
# 2. DATASET LOADING & EXPLORATORY VISUALIZATIONS (EDA)
# ==========================================
df = pd.read_csv('AmesHousing.csv')
print(f"Dataset Shape: {df.shape}")

target_column = 'SalePrice' if 'SalePrice' in df.columns else df.columns[-1]

# --- VISUALIZATION 1: Target Variable Distribution ---
plt.figure(figsize=(10, 5))
sns.histplot(df[target_column], kde=True, color='skyblue', bins=30)
plt.title('Target Distribution: House Sale Price (SalePrice)')
plt.xlabel('Sale Price ($)')
```

```
plt.ylabel('Frequency')
plt.tight_layout()
plt.show()

# --- VISUALIZATION 2: Correlation Heatmap for Top Features ---
plt.figure(figsize=(10, 8))
numeric_df = df.select_dtypes(include=['int64', 'float64'])
top_corr_features = numeric_df.corr()[target_column].abs().sort_values(ascending=False).head(10).index
sns.heatmap(numeric_df[top_corr_features].corr(), annot=True, cmap='Blues', fmt=".2f", linewidths=0.5)
plt.title('Correlation Heatmap: Top 10 Numeric Features with SalePrice')
plt.tight_layout()
plt.show()

# --- VISUALIZATION 3: Feature Relationships (Living Area vs Price) ---
plt.figure(figsize=(9, 5))
sns.scatterplot(data=df, x='Gr Liv Area', y=target_column, alpha=0.6, color='teal')
plt.title('Above Grade Living Area (Gr Liv Area) vs Sale Price')
plt.xlabel('Above Grade Living Area (sq ft)')
plt.ylabel('Sale Price ($)')
plt.tight_layout()
plt.show()

# ==========================================
# 3. PREPROCESSING (Handling Mixed Types)
# ==========================================
X = df.drop(columns=[target_column])
y = df[target_column]

numerical_cols = X.select_dtypes(include=['int64', 'float64']).columns
categorical_cols = X.select_dtypes(include=['object', 'category']).columns

numeric_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='median')),
    ('scaler', StandardScaler())
])

categorical_transformer = Pipeline(steps=[
    ('imputer', SimpleImputer(strategy='constant', fill_value='Missing')),
    ('onehot', OneHotEncoder(handle_unknown='ignore', sparse_output=False))
```

```
])

preprocessor = ColumnTransformer(
    transformers=[
        ('num', numeric_transformer, numerical_cols),
        ('cat', categorical_transformer, categorical_cols)
    ]
)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

X_train_processed = preprocessor.fit_transform(X_train)
X_test_processed = preprocessor.transform(X_test)

encoded_cat_cols = (preprocessor.named_transformers_['cat']
                    .named_steps['onehot']
                    .get_feature_names_out(categorical_cols))
all_feature_names = list(numerical_cols) + list(encoded_cat_cols)

print(f"Processed Training Feature Count: {X_train_processed.shape[1]}")

# ==========================================
# 4. MODEL TRAINING & BENCHMARKING
# ==========================================
models = {
    "Multiple Linear Regression": LinearRegression(),
    "Random Forest Regressor": RandomForestRegressor(n_estimators=100, random_state=42),
    "XGBoost Regressor": XGBRegressor(n_estimators=100, learning_rate=0.1, random_state=42)
}

evaluation_results = []

for name, model in models.items():
    model.fit(X_train_processed, y_train)
    y_pred = model.predict(X_test_processed)

    r2 = r2_score(y_test, y_pred)
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
```

```
    evaluation_results.append({
        "Model": name,
        "R2 Score": r2,
        "MAE": mae,
        "RMSE": rmse
    })

results_df = pd.DataFrame(evaluation_results)
print("\n--- Model Performance Comparison ---")
print(results_df.to_string(index=False))

# --- VISUALIZATION 4: Model Performance Comparison Bar Chart ---
plt.figure(figsize=(9, 5))
sns.barplot(data=results_df, x='Model', y='R2 Score', palette='viridis')
plt.title('Model Benchmarking: Comparison of R² Scores')
plt.ylim(0.8, 1.0)
plt.ylabel('R² Score')
plt.tight_layout()
plt.show()

# ==========================================
# 5. HYPERPARAMETER TUNING & EVALUATION
# ==========================================
print("\nRunning GridSearchCV for XGBoost optimization...")
param_grid = {
    'n_estimators': [100, 200],
    'learning_rate': [0.05, 0.1],
    'max_depth': [3, 5]
}

xgb = XGBRegressor(random_state=42)
grid_search = GridSearchCV(estimator=xgb, param_grid=param_grid, cv=3, scoring='r2', n_jobs=-1)
grid_search.fit(X_train_processed, y_train)

best_xgb = grid_search.best_estimator_
print(f"Best Hyperparameters: {grid_search.best_params_}")

y_pred_tuned = best_xgb.predict(X_test_processed)
```

```
print(f"Tuned XGBoost R2 Score: {r2_score(y_test, y_pred_tuned):.4f}")
print(f"Tuned XGBoost RMSE: {np.sqrt(mean_squared_error(y_test, y_pred_tuned)):.4f}")

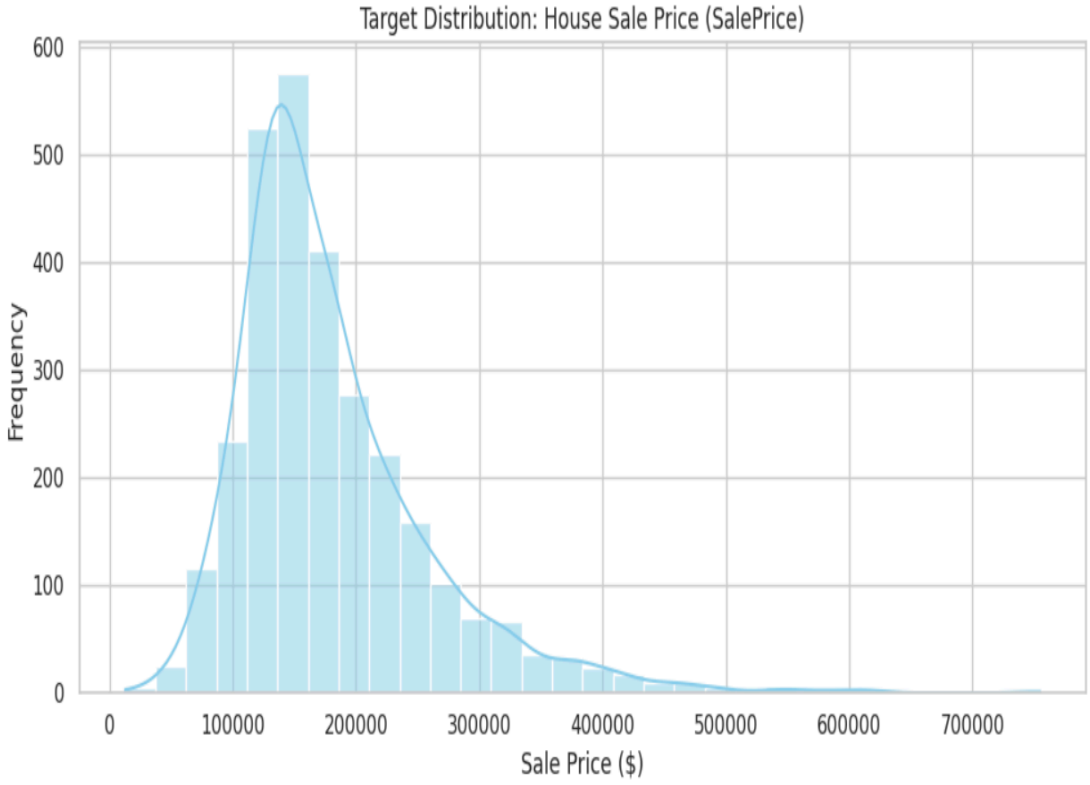
# --- VISUALIZATION 5: Actual vs Predicted Prices ---
plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred_tuned, alpha=0.6, color='crimson')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.title('Actual vs. Predicted Sale Prices (Tuned XGBoost)')
plt.xlabel('Actual Sale Price ($)')
plt.ylabel('Predicted Sale Price ($)')
plt.tight_layout()
plt.show()

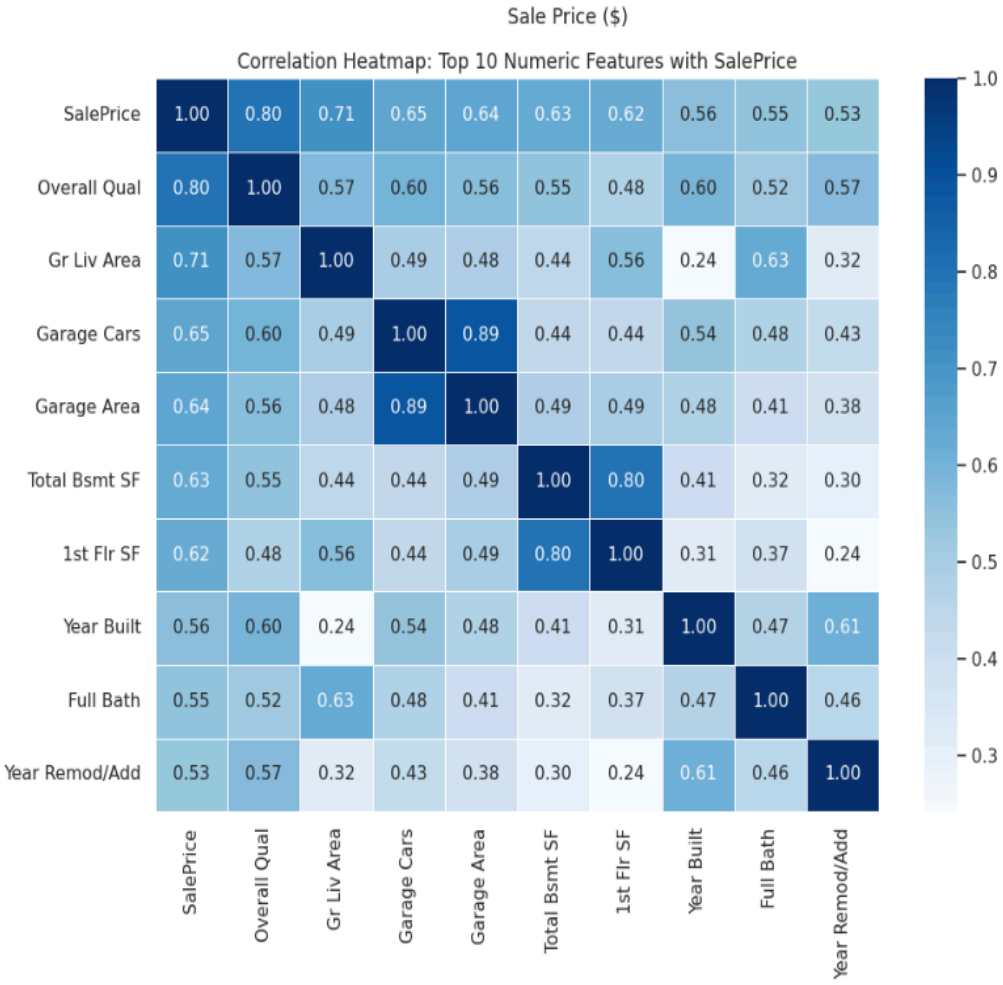
# ==========================================
# 6. FEATURE IMPORTANCE ANALYSIS
# ==========================================
# --- VISUALIZATION 6: Top 10 Feature Importances ---
plt.figure(figsize=(10, 6))
feature_importances = pd.Series(best_xgb.feature_importances_, index=all_feature_names)
feature_importances.nlargest(10).plot(kind='barh', color='darkorange')
plt.title('Top 10 Influential Features for House Price Prediction (XGBoost)')
plt.xlabel('Relative Importance Score')
plt.ylabel('Features')
plt.gca().invert_yaxis()
plt.tight_layout()
plt.show()

# ==========================================
# 7. CROSS-VALIDATION
# ==========================================
cv_scores = cross_val_score(best_xgb, X_train_processed, y_train, cv=3, scoring='r2')
print(f"\nCross-Validation R2 Scores: {cv_scores}")
print(f"Mean CV R2 Score: {cv_scores.mean():.4f}")
```

OUTPUT

```
Dataset Shape: (2930, 82)
```

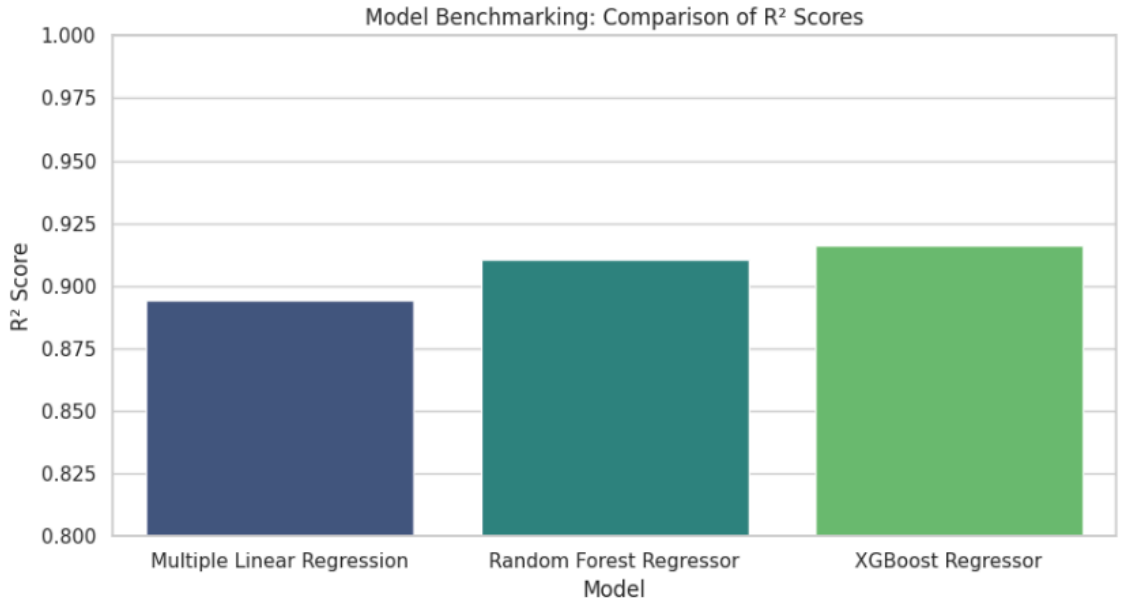






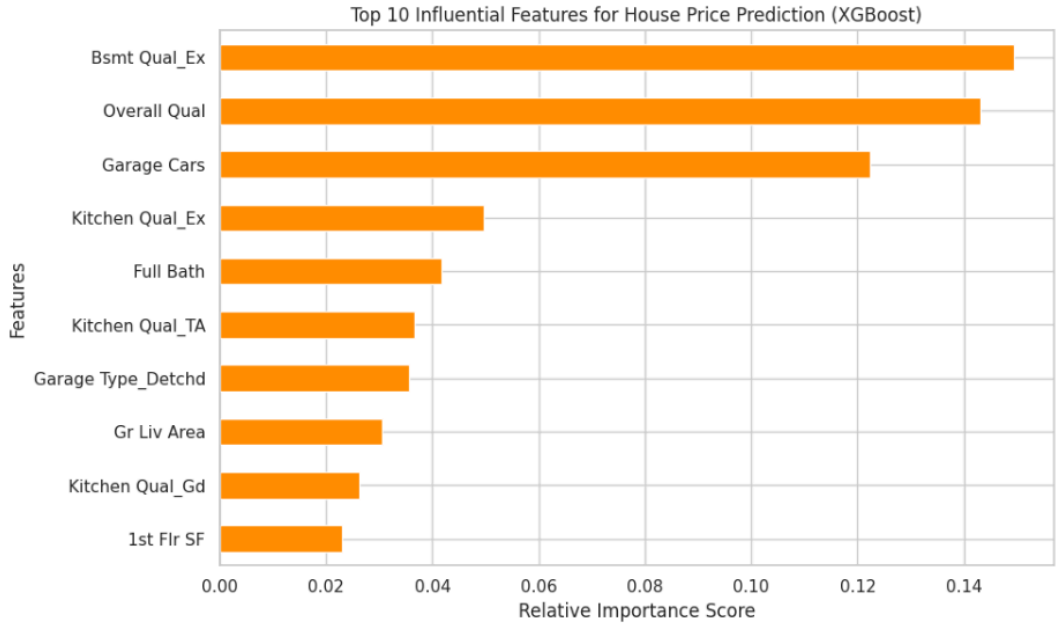

```
Processed Training Feature Count: 317
```

```
--- Model Performance Comparison ---
                      Model  R2 Score          MAE         RMSE
Multiple Linear Regression  0.894146 15710.446326 29132.317480
   Random Forest Regressor  0.910404 15838.429659 26801.920153
         XGBoost Regressor  0.916257 14985.625977 25911.629513
```



```
Running GridSearchCV for XGBoost optimization...
Best Hyperparameters: {'learning_rate': 0.1, 'max_depth': 3, 'n_estimators': 200}
Tuned XGBoost R2 Score: 0.9176
Tuned XGBoost RMSE: 25699.9574
```






```
Cross-Validation R2 Scores: [0.91348541 0.90616018 0.91301912]
Mean CV R2 Score: 0.9109
```